Task 11: Salt-and-Pepper Noise Injection & Restoration

Injects synthetic impulse noise by setting extreme pixel values, followed by image recovery using a median filter.

Clear Answer:

এই প্রোগ্রামটির নাম Salt-and-Pepper Noise Removal using Median Filtering। এটি Digital Image Processing-এর একটি গুরুত্বপূর্ণ Image Restoration Technique।

এই প্রোগ্রামের মূল কাজ হলো:

1. Original Image Load করা।
2. Image-এর মধ্যে Salt-and-Pepper Noise তৈরি করা।
3. Median Filter ব্যবহার করে Noise Remove করা।
4. Original, Noisy এবং Restored Image পাশাপাশি Display করা।

তোমাকে আগের মতোই প্রতিটি Step, Code-এর কাজ এবং Exam-এ শিক্ষককে কীভাবে বুঝাবে তা ব্যাখ্যা করছি।

1. Salt-and-Pepper Noise কী?

Salt-and-Pepper Noise হলো এক ধরনের Image Noise, যেখানে ছবির মধ্যে Random White এবং Black Pixels তৈরি হয়।

এখানে দুই ধরনের Noise থাকে।

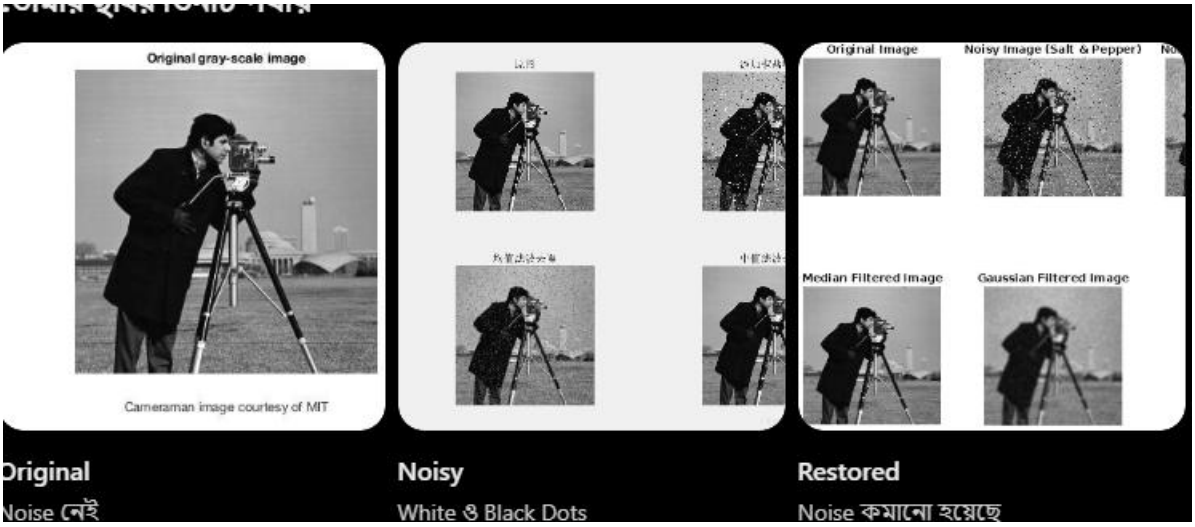
1. Salt Noise:

- ছবির মধ্যে Random White Dots তৈরি হয়।
- Pixel Value হয় 255।
- অর্থাৎ সম্পূর্ণ White Pixel।

2. Pepper Noise:

- ছবির মধ্যে Random Black Dots তৈরি হয়।
- Pixel Value হয় 0।
- অর্থাৎ সম্পূর্ণ Black Pixel।

তোমার ছবির তিনটি পর্যায়



2. Median Filter কী?

Median Filter হলো একটি Non-Linear Filtering Technique, যা ব্যবহার করে Salt-and-Pepper Noise Remove করা হয়।

এখানে প্রতিটি Pixel-এর আশেপাশের Pixel নিয়ে একটি ছোট Window তৈরি করা হয়।

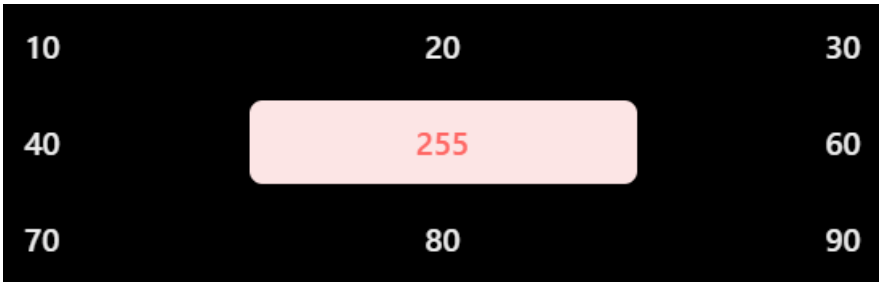
তারপর Window-এর Pixel Value-গুলো ছোট থেকে বড় সাজানো হয় এবং মাঝখানের Value বের করা হয়।

এই Median Value দিয়ে Center Pixel Replace করা হয়।

উদাহরণ:

ধরো, আমাদের একটি 3×3 Matrix আছে।

3×3 Pixel Matrix



Step 1: Pixel Value ছোট থেকে বড় সাজাই।

10, 20, 30, 40, 60, 70, 80, 90, 255

Step 2: মাঝখানের Value বের করি।

60

Step 3: Center Pixel-এর 255-এর জায়গায় 60 বসবে।

এভাবে প্রতিটি Pixel Process করে ছবির Noise কমানো হয়।

3. Complete Corrected Python Code

```
import numpy as np
import matplotlib.pyplot as plt

image = plt.imread("PNG/Task_11_Salt_Pepper_Noise_Restoration.png")

if image.ndim == 3:
    image = (0.299 * image[:, :, 0] +
             0.587 * image[:, :, 1] +
             0.114 * image[:, :, 2])
if image.max() <= 1:
    image = image * 255

image = image.astype(np.uint8)

def add_salt_pepper_noise(image, noise_ratio=0.05):
    noisy_image = image.copy()
    total_pixels = image.size
    num_noise = int(total_pixels * noise_ratio)

    random_positions = np.random.randint(
        0, total_pixels, num_noise
    )

    half = num_noise // 2
    flat_image = noisy_image.flatten()
    flat_image[random_positions[:half]] = 255
    flat_image[random_positions[half:]] = 0
    noisy_image = flat_image.reshape(image.shape)
    return noisy_image
```

```
def median_filter(image, kernel_size=3):
    height, width = image.shape
    padding = kernel_size // 2
    padded_image = np.pad(
        image,
        padding,
        mode='edge'
    )

    filtered_image = np.zeros_like(image)

    for i in range(height):
        for j in range(width):
            window = padded_image[
                i:i + kernel_size,
                j:j + kernel_size
            ]

            values = sorted(window.flatten())
            median_value = values[len(values) // 2]
            filtered_image[i, j] = median_value

    return filtered_image

noisy_image = add_salt_pepper_noise(
    image,
    noise_ratio=0.05
)
restored_image = median_filter(
```

```
    noisy_image,
    kernel_size=3
)
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(noisy_image, cmap='gray')
plt.title("Salt-and-Pepper Noisy Image")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(restored_image, cmap='gray')
plt.title("Restored Image (Median Filter)")
plt.axis("off")

plt.tight_layout()
plt.show()
```

4. Code Explanation (Step by Step)

Step 1: Import Libraries

import numpy as npimport matplotlib.pyplot as plt

এখানে দুটি Library Import করা হয়েছে।

- numpy: Pixel Calculation, Array Operation এবং Random Noise তৈরি করার জন্য।
- matplotlib.pyplot: Original, Noisy এবং Restored Image Display করার জন্য।

Step 2: Load Original Image

image = plt.imread("PNG/Task_11_Salt_Pepper_Noise_Restoration.png")

এখানে plt.imread() ব্যবহার করে নির্দিষ্ট Location থেকে Original Image Load করা হয়েছে।

এখন Image-টি আমাদের Python Program-এর মধ্যে এসেছে।

Step 3: Convert RGB to Grayscale

if image.ndim == 3: image = (0.299 * image[:, :, 0] + 0.587 * image[:, :, 1] + 0.114 * image[:, :, 2])

এখানে প্রথমে Image-এর Dimension Check করা হয়েছে।

image.ndim == 3 মানে Image-টি RGB বা Color Image।

যদি RGB Image হয়, তাহলে সেটিকে Grayscale-এ Convert করা হবে।

এখানে তিনটি Color Channel ব্যবহার করা হয়েছে:

- Red = 0.299
- Green = 0.587
- Blue = 0.114

এই তিনটি Channel-এর Weighted Sum ব্যবহার করে Grayscale Image তৈরি করা হয়েছে।

Step 4: Normalize Pixel Values

if image.max() <= 1: image = image * 255image = image.astype(np.uint8)

এখানে Pixel Value-এর Range Check করা হয়েছে।

কিছু Image-এর Pixel Value 0 থেকে 1-এর মধ্যে থাকে।

সেক্ষেত্রে image * 255 ব্যবহার করে Pixel Value-কে 0 থেকে 255 Range-এ নিয়ে আসা হয়।

এরপর astype(np.uint8) ব্যবহার করে Pixel Value-কে 8-bit Integer Format-এ Convert করা হয়েছে।

মনে রাখবে: Grayscale Image-এর Pixel Value সাধারণত 0 থেকে 255 পর্যন্ত হয়।

Step 5: Create Salt-and-Pepper Noise Function

def add_salt_pepper_noise(image, noise_ratio=0.05):

এখানে add_salt_pepper_noise() নামে একটি Function তৈরি করা হয়েছে।

এই Function-এর কাজ হলো Original Image-এর মধ্যে Random White এবং Black Noise তৈরি করা।

এখানে:

- image = Original Image।
- noise_ratio=0.05 = মোট Pixel-এর 5% Noise তৈরি হবে।

Step 6: Copy Original Image

noisy_image = image.copy()

এখানে Original Image-এর একটি Copy তৈরি করা হয়েছে।

কারণ আমরা Original Image পরিবর্তন করতে চাই না।

আমরা Copy Image-এর মধ্যে Noise যোগ করব।

Step 7: Calculate Total Pixels and Noise

total_pixels = image.sizenum_noise = int(total_pixels * noise_ratio)

এখানে দুটি Calculation করা হয়েছে।

প্রথম লাইনে Image-এর মোট Pixel সংখ্যা বের করা হয়েছে।

দ্বিতীয় লাইনে কতগুলো Pixel-এ Noise তৈরি হবে তা Calculate করা হয়েছে।

উদাহরণ:

ধরো, Image-এ মোট 100,000 Pixels আছে।

Noise Ratio = 0.05

\[100000 \times 0.05 = 5000 \]

অর্থাৎ মোট 5,000টি Pixel-এ Noise তৈরি হবে।

Step 8: Generate Random Positions

random_positions = np.random.choice(total_pixels, num_noise, replace=False)

এখানে np.random.choice() ব্যবহার করে Random Pixel Position নির্বাচন করা হয়েছে।

- total_pixels = মোট Pixel সংখ্যা।
- num_noise = কতটি Position নির্বাচন হবে।
- replace=False = একই Position একাধিকবার নির্বাচন হবে না।

এই Position-গুলোতেই Noise যোগ করা হবে।

Step 9: Divide Noise into Two Parts

half = num_noise // 2flat_image = noisy_image.flatten()

প্রথম লাইনে মোট Noise-এর অর্ধেক বের করা হয়েছে।

দ্বিতীয় লাইনে Image-কে 1D Array-তে Convert করা হয়েছে।

flatten() ব্যবহার করে 2D Image Array-কে Single Dimension-এ নেওয়া হয়।

এতে Random Position অনুযায়ী Pixel পরিবর্তন করা সহজ হয়।

Step 10: Add Salt Noise

flat_image[random_positions[:half]] = 255

এখানে Random Position-এর প্রথম অর্ধেক Pixel-এর Value 255 করা হয়েছে।

255 মানে White Pixel।

এভাবেই Image-এর মধ্যে Salt Noise তৈরি হয়।

Step 11: Add Pepper Noise

flat_image[random_positions[half:]] = 0

এখানে Random Position-এর বাকি Pixel-এর Value 0 করা হয়েছে।

0 মানে Black Pixel।

এভাবেই Image-এর মধ্যে Pepper Noise তৈরি হয়।

Step 12: Reshape Image

noisy_image = flat_image.reshape(image.shape)return noisy_image

flatten() ব্যবহার করে আমরা Image-কে 1D Array করেছিলাম।

এখন reshape() ব্যবহার করে আবার আগের 2D Shape-এ ফিরিয়ে আনা হয়েছে।

return noisy_image দিয়ে Noise যুক্ত Image-টি Function থেকে Return করা হয়েছে।

Step 13: Create Median Filter Function

def median_filter(image, kernel_size=3):

এখানে median_filter() নামে একটি Function তৈরি করা হয়েছে।

এই Function-এর কাজ হলো Noisy Image থেকে Salt-and-Pepper Noise Remove করা।

এখানে:

- image = Noisy Image।
- kernel_size=3 = 3×3 Kernel ব্যবহার করা হবে।

Step 14: Get Image Dimensions and Padding

height, width = image.shapepadding = kernel_size // 2

প্রথম লাইনে Image-এর Height এবং Width বের করা হয়েছে।

দ্বিতীয় লাইনে Kernel-এর Radius বা Padding Size Calculate করা হয়েছে।

যেহেতু Kernel Size 3:

\[Padding = 3 // 2 = 1 \]

অর্থাৎ চারপাশে এক Pixel করে Padding দেওয়া হবে।

Step 15: Apply Padding

padded_image = np.pad(image, padding, mode='edge')

এখানে np.pad() ব্যবহার করে Image-এর চারপাশে Padding দেওয়া হয়েছে।

mode='edge' ব্যবহার করার কারণে Image-এর Boundary Pixel-গুলো Replicate করা হয়।

কেন Padding প্রয়োজন?

Image-এর Edge বা Corner-এর Pixel Process করার সময় 3×3 Kernel-এর সম্পূর্ণ Neighborhood পাওয়া যায় না।

Padding ব্যবহার করলে Boundary Pixel-ও সহজে Process করা যায়।

Step 16: Create Empty Filtered Image

filtered_image = np.zeros_like(image)

এখানে Original Image-এর মতো একই Shape-এর একটি Blank Image তৈরি করা হয়েছে।

np.zeros_like() ব্যবহার করার কারণে সব Pixel-এর Initial Value হবে 0।

আমরা Median Value দিয়ে এই Image-এর Pixel পূরণ করব।

Step 17: Visit Every Pixel

for i in range(height): for j in range(width):

এখানে Nested Loop ব্যবহার করা হয়েছে।

প্রথম Loop প্রতিটি Row Access করে।

দ্বিতীয় Loop প্রতিটি Column Access করে।

অর্থাৎ Image-এর প্রতিটি Pixel একে একে Process করা হবে।

Step 18: Extract 3×3 Neighborhood

window = padded_image[i:i + kernel_size, j:j + kernel_size]

এটি Median Filtering-এর গুরুত্বপূর্ণ অংশ।

এখানে Current Pixel-এর আশেপাশের 3×3 Window নেওয়া হয়েছে।

এই Window-এর মধ্যে মোট 9টি Pixel থাকবে।

এই 9টি Pixel-এর Value ব্যবহার করে Median বের করা হবে।

Step 19: Sort Pixel Values

values = sorted(window.flatten())

এখানে দুটি কাজ করা হয়েছে।

- flatten() দিয়ে 3×3 Matrix-কে 1D List-এ Convert করা হয়েছে।
- sorted() দিয়ে Pixel Value-গুলো ছোট থেকে বড় সাজানো হয়েছে।

উদাহরণ:

10, 20, 30, 40, 60, 70, 80, 90, 255

Step 20: Find Median Value

median_value = values[len(values) // 2]

এখানে Sorted Pixel Value-এর মাঝখানের Value বের করা হয়েছে।

যেহেতু 3×3 Kernel-এ 9টি Pixel থাকে:

\[Median = 5th\ Value \]

অর্থাৎ সাজানো 9টি Value-এর মাঝখানের Value-টি Median।

Step 21: Replace Center Pixel

filtered_image[i, j] = median_value

এখানে Calculate করা Median Value দিয়ে Output Image-এর Current Pixel Replace করা হয়েছে।

এই Process প্রতিটি Pixel-এর জন্য Repeat হবে।

ফলে Noisy Image থেকে Noise কমে Restored Image তৈরি হবে।

Step 22: Return Restored Image

return filtered_image

এখানে সম্পূর্ণ Filtered Image Function থেকে Return করা হয়েছে।

Step 23: Add Noise to Original Image

noisy_image = add_salt_pepper_noise(image, noise_ratio=0.05)

এখানে add_salt_pepper_noise() Function Call করা হয়েছে।

Original Image-এর 5% Pixel-এ Random Salt-and-Pepper Noise যোগ করা হয়েছে।

Step 24: Apply Median Filter

restored_image = median_filter(noisy_image, kernel_size=3)

এখানে Noisy Image-এর ওপর Median Filter Apply করা হয়েছে।

3×3 Kernel ব্যবহার করে Noise Remove করা হয়েছে।

Filtered Result restored_image Variable-এ Store হয়েছে।

Step 25: Display Three Images

plt.figure(figsize=(15, 5))

এখানে Display Window-এর Size নির্ধারণ করা হয়েছে।

plt.subplot(1, 3, 1)

এখানে 1 Row এবং 3 Column-এর Layout তৈরি করা হয়েছে।

তিনটি Image Display করা হয়েছে:

1. Original Image
2. Salt-and-Pepper Noisy Image
3. Restored Image (Median Filter)

সবশেষে:

plt.tight_layout()plt.show()

plt.tight_layout() Image-গুলোর মধ্যে Spacing ঠিক করে।

plt.show() সব Image পাশাপাশি Display করে।

বাংলায় ব্যাখ্যা:

প্রথমে আমি প্রয়োজনীয় Library Import করেছি এবং Original Image Load করেছি। এরপর RGB Image হলে সেটিকে Grayscale-এ Convert করেছি এবং Pixel Value 0 থেকে 255 Range-এ নিয়ে এসেছি।

তারপর add_salt_pepper_noise() নামে একটি Function তৈরি করেছি। এই Function-এর মাধ্যমে Random Pixel Position নির্বাচন করে কিছু Pixel-এর Value 255 এবং কিছু Pixel-এর Value 0 করেছি। ফলে Image-এর মধ্যে Salt-and-Pepper Noise তৈরি হয়েছে।

এরপর median_filter() নামে আরেকটি Function তৈরি করেছি। এখানে 3×3 Kernel ব্যবহার করে প্রতিটি Pixel-এর আশেপাশের 9টি Pixel Value সংগ্রহ করেছি। তারপর Value-গুলো ছোট থেকে বড় সাজিয়ে Median Value বের করেছি।

সবশেষে Median Value দিয়ে Center Pixel Replace করেছি। এই Process প্রতিটি Pixel-এর জন্য Repeat করে Noisy Image থেকে Noise কমিয়েছি এবং Restored Image তৈরি করেছি।

এরপর Original Image, Noisy Image এবং Restored Image পাশাপাশি Display করেছি।

Copy Exam Explanation

6. Important Viva Questions and Answers

Q1. What is Salt-and-Pepper Noise?

Answer: Salt-and-Pepper Noise is a type of image noise that creates random white and black pixels in an image.

বাংলা: ছবির মধ্যে Random White এবং Black Pixel তৈরি হওয়াকে Salt-and-Pepper Noise বলে।

Q2. What is Salt Noise?

Answer: Salt Noise represents white pixels with intensity value 255.

Q3. What is Pepper Noise?

Answer: Pepper Noise represents black pixels with intensity value 0.

Q4. What is Median Filtering?

Answer: Median Filtering is a non-linear technique that removes noise by replacing the center pixel with the median value of its neighboring pixels.

Q5. Why do we use a 3×3 Kernel?

Answer: We use a 3×3 Kernel to examine 9 neighboring pixels around the center pixel.

Q6. Why do we use flatten()?

Answer: It converts a 2D array into a 1D array.

Q7. What does sorted() do?

Answer: It arranges pixel values in ascending order.

Q8. What is the purpose of np.pad()?

Answer: It adds padding around the image to process boundary pixels properly.

Q9. Why do we use Median instead of Mean?

Answer: Median Filtering is more effective at removing Salt-and-Pepper Noise and generally preserves image edges better than Mean Filtering.

Q10. What does noise_ratio=0.05 mean?

Answer: It means approximately 5% of the image pixels will be selected for noise addition.

Q11. What is np.zeros_like()?

Answer: It creates a new array with the same shape and data type as the original array, filled with zeros.

Q12. Why do we use image.copy()?

Answer: To preserve the original image while creating a noisy version.

7. Complete Working Process



Key Caveats:

1. noise_ratio=0.05 মানে মোট Pixel-এর প্রায় 5% Noise হবে।
2. kernel_size=3 হলে মোট 9টি Pixel নিয়ে Median বের করা হয়।
3. Median Filtering Salt-and-Pepper Noise-এর জন্য কার্যকর।
4. Original Code-এর np.random.randint() একই Pixel Position একাধিকবার নির্বাচন করতে পারে। Corrected Code-এ np.random.choice(..., replace=False) ব্যবহার করা হয়েছে।
5. Median Filtering Noise কমালেও খুব বেশি Noise থাকলে কিছু Fine Details হারাতে পারে।

পরীক্ষার জন্য সবচেয়ে গুরুত্বপূর্ণ ৫টি Code Line:

num_noise = int(total_pixels * noise_ratio)flat_image[random_positions[:half]] = 255flat_image[random_positions[half:]] = 0values = sorted(window.flatten())median_value = values[len(values) // 2]

এই পাঁচটি লাইন বুঝতে পারলে Noise তৈরি করা থেকে শুরু করে Median বের করে Noise Remove করার মূল Concept বুঝতে পারবে।